Contents

Introduction **3**

1 Literature Review **7**

1.1 Computational humor as a research problem 7
1.2 Evolution of approaches to humor generation 8
1.3 The issues of humor evaluation 9
1.4 The promise of reasoning for humor generation 10
1.5 Why standard post-training is a poor fit for humor 11
1.6 Reasoning-oriented post-training 12
1.7 Research gap 13

2 Methodology **14**

2.1 Research scope and implementation status 14
2.2 Corpus construction 14
2.3 Dense embedding generation 15
2.4 Keyword extraction as weak prompt approximation 16
2.5 Reference construction by keyword grouping and retrieval 17
2.5.1 Keyword grouping and prompt rendering 17
2.5.2 Embedding-based reference retrieval 17
2.5.3 Deduplication and split construction 19
2.6 Candidate generation pipeline 19
2.7 Implemented automatic evaluation 20
2.7.1 Pairwise comparison setup 20
2.7.2 Leaderboard aggregation 20
2.8 Theoretical framework for MRVF training 21
2.8.1 Notation 21
2.8.2 RL with verifiable rewards 21
2.8.3 VeriFree 22

2.8.4 Multi-reference VeriFree 22
2.8.5 Incomplete observed references 23
2.8.6 Proposed training loop 24
2.9 Evaluation protocol for the preliminary thesis 25

3 Results **26**
3.1 Overview of the implemented pilot 26
3.2 Pilot dataset statistics 27
3.3 Reference-set quality 28
3.4 Qualitative examples 29
3.5 Interpretation of the pilot as a multi-reference dataset 30
3.6 Limitations revealed by the current implementation 31
3.6.1 Prompt quality is approximate 31
3.6.2 Coverage is still partial 31
3.6.3 Reference incompleteness remains unresolved 32
3.6.4 No committed benchmark outputs yet 32
3.6.5 Training remains future work 32
3.7 What the pilot already contributes 32
3.8 Next stage of the study 33

Conclusion **35**

References **37**

A Implemented Prompt Templates **41**
A.1 Reference-generation prompt 41
A.2 Embedding query instructions 41
A.3 Evaluation prompt 42

B Supplementary Notes on the Software Architecture **43**

Introduction

Humor generation is one of the clearest examples of a language task that matters precisely because it is hard to verify automatically. A joke may be grammatically correct, semantically coherent, and even topically relevant while still failing at the central target: making a reader laugh. This makes humor generation a useful stress test for modern language-model post-training. In domains such as mathematics, coding, or formal logic, reinforcement learning can rely on explicit verification signals. In humor, by contrast, the output space is open-ended, the number of acceptable answers is large, and human judgments are noisy. The same prompt can admit many good jokes, while a dataset usually records only one or a few. These properties make humor an informative case for studying reasoning in non-verifiable domains.

The relevance of the topic is determined by three converging trends. First, recent surveys show that computational humor is moving from isolated rule-based or template-based systems toward large language model pipelines, yet the gap between fluent text and genuinely funny output remains substantial [Loakman et al., 2025a]. Second, post-training research increasingly suggests that supervised fine-tuning (SFT) and reinforcement learning optimize different capabilities: SFT is effective at imitation, whereas reinforcement learning is more promising when generalization beyond the training surface is required [Chu et al., 2025]. Third, reasoning-focused reinforcement learning has recently produced strong results in domains with explicit verifiers, motivating the question of whether similar methods can be adapted to creative tasks where direct verification is impossible [DeepSeek-AI, 2025, Zhou et al., 2025, Yu et al., 2025].

This thesis studies humor generation as a non-verifiable reasoning problem. The central idea is that humor does not only require a good final answer; it often benefits from exploratory reasoning over premises, associations, contrasts, and punchline formats. Multi-step humor systems already exploit this fact at inference time by explicitly decomposing the creative process into intermediate

stages [Tikhonov and Shtykovskiy, 2024, Zhang et al., 2025, Kim and Chilton, 2025]. However, such systems are architecturally restrictive: the number of stages, the reasoning style, and the intermediate control signals are designed manually. The alternative investigated in this thesis is to construct a training setting in which a model can learn how to reason for humor generation without requiring a symbolic verifier.

The object of the study is the post-training of large language models for creative text generation. The subject of the study is the design of a multi-reference verifier-free reinforcement learning objective for humor generation together with the supporting data pipeline needed to make such an objective operational.

The goal of the thesis is to develop a preliminary methodological and software foundation for training humor-generation models with verifier-free reinforcement learning, while remaining faithful to the actual implementation status of the project. In practical terms, the work aims to transform a large joke corpus into prompt-reference sets suitable for multi-reference training and evaluation, formulate the corresponding reinforcement learning objective, and analyze whether the constructed data artifacts are adequate for a later MRVF training stage.

To achieve this goal, the following tasks are addressed:

1. analyze recent literature on computational humor, post-training, and verifier-free reasoning methods;

2. define the research gap between current humor pipelines and reinforcement learning for non-verifiable domains;

3. build a corpus-processing pipeline that converts raw jokes into weak prompts with multiple retrieved references;

4. implement candidate-generation and evaluation components for later benchmark comparisons;

5. formulate a multi-reference extension of verifier-free reinforcement learning and discuss its statistical consequences under incomplete references;

6. evaluate the current pilot data pipeline using descriptive statistics, qualitative examples, and implementation analysis;

7. outline the next experimental stage needed to turn the pilot into a full thesis-level training study.

The working hypothesis of the thesis is that humor generation can be made more amenable to reasoning-oriented post-training if the learning problem is reframed from "predict the single gold joke" to "assign probability mass to a set of acceptable jokes conditioned on a weak prompt." Under this hypothesis, the main obstacle is not only reward design, but also dataset structure: a useful training signal requires prompts that admit multiple valid references rather than isolated one-to-one joke mappings.

The scientific novelty of the work is threefold. First, it adapts the verifier-free reinforcement learning perspective of Zhou et al. [2025] and Yu et al. [2025] to a creative task where multiple valid outputs are expected by definition. Second, it makes explicit the difference between the original conceptual report and the current implemented system: rather than a full generative prompt inversion module, the present codebase realizes a retrieval-based approximation built on keywords, embedding search, and reference-set construction. Third, it provides an empirical pilot analysis of this approximation on real data, which is necessary before a more expensive reinforcement-learning stage can be justified.

The practical value of the work lies in the implemented software pipeline. The repository currently contains modular Python components for corpus collection, embedding generation, keyword extraction, reference retrieval, candidate generation, and pairwise evaluation. These components produce reusable parquet datasets and isolate stages that can later be reused for baseline generation, preference comparison, and reinforcement-learning experiments.

The empirical basis of the work consists of the combined joke corpus built from the Short Jokes and rJokes collections. In the current local state of the repository, the complete raw corpus contains 664,048 jokes, while the downstream pilot pipeline processes a smaller subset for prompt construction and reference analysis. This distinction is important: the thesis is preliminary not only in conceptual scope, but also in the current scale of the implemented experiments.

Methodologically, the thesis combines literature review, software engineering, corpus construction, embedding-based retrieval, mathematical derivation of an objective function, and descriptive analysis of intermediate artifacts. This combination reflects the actual status of the research problem: before large-scale

reinforcement learning can be defended, the data representation, prompt formulation, and reference structure must first be stabilized.

The structure of the thesis follows the standard university requirements. Chapter 1 reviews the literature on humor generation, evaluation, and reasoning-oriented post-training. Chapter 2 presents the methodology, including the implemented pipeline and the formal MRVF objective. Chapter 3 reports the current pilot results, discusses strengths and limitations of the constructed dataset, and explains what remains to be implemented for the final thesis. The work concludes with a summary of findings, followed by the list of references and appendices.

Chapter 1

Literature Review

1.1 Computational humor as a research problem

Humor has long been treated as a hard case for natural language processing because successful jokes depend on more than grammatical well-formedness. They also depend on incongruity, shared knowledge, timing, and audience expectations. Early reviews already described humor as one of the more complex targets for artificial intelligence, noting that implemented systems were largely confined to narrow punning mechanisms and that broader humor processing would require substantial world knowledge and reasoning [Ritchie, 2001]. Two later surveys reach a similar conclusion from different angles. Amin and Burghardt [2020] show that computational humor generation has historically traded breadth for control: template- and rule-based systems could work well with identifiable humorous mechanisms, but only within highly restricted domains. More recently, Loakman et al. [2025a] and Lemmens and De Marez [2026] argue that the large-language-model era has broadened coverage while leaving the central problem unresolved, since fluent text is not equivalent to humor that readers actually find funny.

A further difficulty is theoretical pluralism. Humor theory offers several recurring explanatory families—most notably incongruity-based accounts, but also superiority and relief traditions—yet no single framework exhausts the phenomenon. This matters computationally because different joke types rely on different mechanisms. Kao et al. [2016], for instance, formalize incongruity in puns as ambiguity plus distinctiveness and show that these variables predict human funniness judgments. Their work is important not only because it finds an application for a classical humor theory, but also because it illustrates the limits of narrow formalization: a model that explains homophonic puns does not automat-

ically explain narrative jokes, topical jokes, or socially situated humor. Recent theory-driven work in humor detection starts from the same premise, namely that no single comprehensive humor theory is sufficient and that computational models typically learn only fragments of humor [De Marez et al., 2024]. For the present thesis, the consequence is methodological: humor should be treated as a creative task with a multitude of potential mechanisms rather than as a single scalar attribute of communication.

1.2 Evolution of approaches to humor generation

Before the rise of large language models, humor generation systems were usually narrow by design. As summarized by Amin and Burghardt [2020], earlier work often relied on hand-written templates, lexical resources, phonetic transformations, or tightly constrained generation procedures. These systems had an important advantage: they encoded at least one explicit humorous mechanism. Their disadvantage was low adaptability. They produced humor only under carefully chosen linguistic conditions and therefore did not scale well beyond punning, stock joke forms, or limited domains.

Large language models invert that trade-off. They can produce open-ended humorous text over a far broader range of prompts, but fluency does not guarantee robust humor competence. This is visible in recent large-scale evaluations. Zhang et al. [2024] introduce a multimodal cartoon-captioning benchmark built from more than 250 million human ratings over more than 2.2 million captions, and show that current fine-tuning methods remain limited on creative tasks. Even strong systems such as GPT-4 and Claude remain below top human contestants. Humor-specific understanding studies point in the same direction. Xu et al. [2024] show that large language models exhibit what they call lazy pun generation: instead of fusing two meanings into a single compact pun, the model frequently states both related words or meanings separately, which satisfies the task superficially. This finding is important methodologically, because it shows that simple lexical-success metrics can overestimate pun ability. Evaluation must also distinguish structural quality and originality from mechanically assembled or copied wordplay. Cocchieri et al. [2025] construct a contamination-resistant humor benchmark and find that most language models still underperform humans

on pun comprehension, resolution, and generation. Going beyond puns, Loakman et al. [2025b] show that current models, including reasoning models, do not reliably explain all joke types, which highlights how much of the existing literature remains focused on relatively simple joke forms.

Taken together, these studies suggest that current models are better at reproducing the surface form of familiar jokes than at generalizing across humor mechanisms. This distinction is important for the present thesis. A system can be good at producing joke-like text without being good at humor in the stronger sense required by open-ended generation.

1.3 The issues of humor evaluation

Evaluation is a central difficulty in computational humor. Humor is subjective, but the problem is not only subjectivity in the colloquial sense. What counts as a good joke depends on audience, context, delivery, and the dimension being evaluated. Even dataset papers emphasize that humor varies across topic, time, and recipient, making it difficult to identify a single target variable [Weller and Seppi, 2020]. For this reason, recent humor work increasingly avoids treating funniness as a scalar metric.

The clearest example is Sakabe et al. [2025], who evaluate Oogiri responses on six dimensions: novelty, clarity, relevance, intelligence, empathy, and overall funniness. Their results show that current language models generate responses between low- and mid-tier human performance, but they notably lag in empathy. The authors further argue that this deficit helps explain why model-based evaluation does not reproduce human humor assessment well. The significance of this result is twofold. First, it shows that humor quality is not reducible to "is this funny?" Second, it shows that human and model judgments may privilege different qualities.

A complementary perspective comes from Winters and Van der Stockt [2025], who compare human comedians and GPT-4 in a live improvisational setting rather than in an offline text-only benchmark. Their study is valuable because it isolates a stronger comparison point: both humans and the model must respond rapidly to the same audience suggestions. Human jokes are still preferred slightly more often, but the margin is not overwhelming, and the design highlights how

much delivery and situational context affect humor evaluation. This is a useful corrective to benchmark-only thinking. A text that performs well in static judging is not necessarily the text that works best in real comedic interaction.

These findings also clarify the role of LLM-as-a-judge methods. Structured evaluation frameworks such as Hashemi et al. [2024] are useful because they replace one-dimensional scoring with explicit rubrics and calibration. However, that paper also shows that raw LLM rubric judgments do not automatically align well with human ratings. For humor generation, this implies that LLM judges are best treated as measurement instruments with a limited domain of validity, not as transparent replacements for human preference.

1.4 The promise of reasoning for humor generation

One of the most important shifts in recent humor-generation research is the move from one-shot continuation to explicit reasoning or staged creative search. At the prompting level, Chen et al. [2023] model humor generation through step-by-step comedy-writing instructions for GPT-3. Zhong et al. [2024] go further by arguing that standard chain-of-thought prompting is poorly matched to creative tasks, and propose a Leap-of-Thought paradigm for Oogiri-style humor generation that emphasizes associative jumps rather than strictly sequential decomposition.

More explicit generation systems strengthen this line of work. Tikhonov and Shtykovskiy [2024] reconstruct one-liner joke writing as a multistep process and report improvements over zero-shot GPT-4 and other baselines in human evaluation. Kim and Chilton [2025] frame humor generation as the combination of cognitive, social, and creative skills. In their meme-captioning study, skill-augmented systems substantially outperform large language model baselines and approach top-rated human captions for the target audience. In multimodal settings, Zhang et al. [2025] propose a theory-guided multi-stage reasoning framework intended to improve both interpretability and humor quality.

These papers are important because they collectively reject the assumption that humor is best produced by an instant completion. Instead, they model humor generation as a search process over premises, associations, reframings, and audience-sensitive choices, similar to how human experts approach comedy

writing or any creative tasks. An informational gap between the premise and the joke which is only filled out during inference that may be latent. At the same time, they share a limitation that is directly relevant to the present thesis: the structure of the reasoning process is defined rigidly. The number of stages, the type of intermediate representations, and the order of operations are fixed by the researcher. Such systems are therefore strong demonstrations of the usefulness of reasoning, but weaker evidence that a model can learn for itself when and how to reason for humor.

1.5 Why standard post-training is a poor fit for humor

The most straightforward route to improving humor generation is supervised fine-tuning on large joke corpora. In practice, however, that objective is a poor match for the structure of the task. Joke datasets contain many repeated setups, familiar wordplay patterns, and conventional punchline forms. As a result, supervised fine-tuning can improve stylistic plausibility without necessarily improving creative generalization. Large humor resources such as rJokes are valuable for scale and topical variety, but they were not constructed as multi-reference generation datasets in which many distinct outputs can simultaneously be correct [Weller and Seppi, 2020].

Recent literature strengthens this concern from two directions. First, humor-specific work shows that even very large preference datasets do not make creative alignment easy. Zhang et al. [2024] explicitly highlight the limitations of RLHF- and DPO-style fine-tuning on creative captioning. Second, post-training work outside humor makes the memorization-generalization distinction sharper. Chu et al. [2025] show that reinforcement learning generalizes more effectively than supervised fine-tuning to unseen textual and visual variants, whereas supervised fine-tuning tends to memorize training patterns and struggle out of distribution. This contrast is especially consequential in humor generation, where a weak prompt can have many good continuations and where memorizing frequent joke shells is not the same as producing an apt and surprising punchline.

Preference optimization has its own difficulties. RLHF assumes that a reward model can learn a stable proxy for the target behavior, but humor judg-

ments are noisy and heterogeneous. Gao et al. [2023] show more generally that aggressively optimizing an imperfect reward model can harm true performance. Humor-specific evaluation suggests that the risk is amplified in creative domains: Sakabe et al. [2025] show systematic divergence between human and model humor judgments, while Winters and Van der Stockt [2025] show that humor quality depends on aspects of performance that are hard to capture in offline textual reward signals. For these reasons, neither supervised fine-tuning nor standard preference optimization cleanly solves the central problem.

1.6 Reasoning-oriented post-training

The recent success of reasoning-oriented post-training has come mainly from domains with objective verification. Wei et al. [2022] show that intermediate reasoning can substantially change model behavior on tasks that reward multi-step inference. Lightman et al. [2023] extend this logic from prompting to supervision, showing that process supervision outperforms outcome-only supervision on the MATH benchmark. DeepSeek-AI [2025] then demonstrate that reinforcement learning can elicit long-form reasoning without relying on human-authored chains of thought, and the clearest gains are reported in verifiable domains such as mathematics, coding, and related STEM tasks.

This creates the central transfer problem for humor. Humor may also benefit from extended reasoning, yet there is usually no deterministic verifier that can declare a joke correct. Recent verifier-free approaches aimed at domains outside of mathematics and coding attempt to close this gap by replacing the explicit verifiable reward with intrinsic probabilities assigned to reference answers. Zhou et al. [2025] propose VeriFree and show that, under a single-reference assumption, maximizing the conditional probability of the reference answer given the prompt and reasoning trace is equivalent to maximizing an exact-match verifier objective in expectation. Yu et al. [2025] develop a related approach named RLPR where reward is formulated as a mean likelihood over individual tokens to enhance variance stability.

For humor, however, the single-reference assumption is not a minor technical detail, it is the wrong data model. Prompts such as “write a joke about a frog” inherently admit many plausible responses, and any dataset will observe only a

small and biased subset of them. In other words, humor is both multi-reference and only partially observed. This means that verifier-free reasoning methods are conceptually relevant to humor generation, but not directly applicable without reformulating the reward around sets of references rather than single targets.

1.7 Research gap

The literature reviewed above leaves a precise gap. Humor-generation studies increasingly show that staged reasoning, skill decomposition, or theory-guided generation can improve outputs, but these systems mostly rely on rigid inference pipelines [Chen et al., 2023, Zhong et al., 2024, Tikhonov and Shtykovskiy, 2024, Kim and Chilton, 2025, Zhang et al., 2025]. Work on reasoning-oriented reinforcement learning shows that useful reasoning behaviors can be elicited, but the strongest existing formulations assume either objective verifiability or a single reference answer [Lightman et al., 2023, DeepSeek-AI, 2025, Zhou et al., 2025, Yu et al., 2025]. Meanwhile, humor evaluation research shows that funniness is multidimensional, context-sensitive, and only imperfectly captured by model judges [Zhang et al., 2024, Sakabe et al., 2025, Winters and Van der Stockt, 2025, Hashemi et al., 2024].

The present thesis is positioned at the intersection of these findings. It claims that humor benefits from reasoning, but rejects the need to formulate a fixed creative script. It also expands on verifier-free reinforcement-learning, but argues that creative tasks require a multi-reference formulation because multiple outputs can be simultaneously good and the observed reference set is incomplete. The resulting research question is therefore whether verifier-free reasoning objectives can be adapted to partially observed multi-reference humor generation in a way that encourages creative search without collapsing into memorized training samples.

Chapter 2

Methodology

2.1 Research scope and implementation status

The methodology of this preliminary thesis has two layers. The first layer is an *implemented data and evaluation pipeline*, already present in the repository. The second layer is a *proposed training objective*, namely multi-reference verifier-free reinforcement learning (MRVF), which is formulated mathematically but not yet implemented in code. This distinction is important for the scope of the present work. The repository already contains pipelines for corpus construction, embedding generation, keyword extraction, reference retrieval, candidate generation, and automatic pairwise evaluation. However, the training stage itself remains future work. Consequently, this chapter distinguishes between what is currently operational in the codebase and what is proposed as the next stage of the project.

At a high level, the implemented system constructs a weakly supervised multi-reference dataset from raw joke corpora. Jokes are first merged into a unified corpus, embedded with a dense retrieval model, and reduced to a small set of salient keywords. These keywords are then expanded into one-keyword and two-keyword prompt groups, which are used to retrieve semantically related joke references from the embedding index. The resulting prompt–reference structure supports two immediate uses. First, it provides a pilot substrate for baseline generation and automatic evaluation. Second, it defines the reference sets required by the proposed MRVF training objective.

2.2 Corpus construction

The empirical basis of the project is a merged corpus built from two public joke collections: the *Short Jokes* dataset and the *rJokes* dataset. In the repository,

this stage is implemented in `src/pipelines/jokes.py`. The pipeline downloads the raw files from pinned source revisions, converts them into parquet format, and preserves source metadata for each joke. For every row, the stored fields are `id`, `text`, `source_name`, `source_filename`, and `source_id`. After preprocessing the two corpora separately, they are concatenated and assigned a new global integer identifier.

This design is intentionally conservative. The pipeline does not perform aggressive text normalization beyond removing empty entries and standardizing storage format. For humor, such caution is desirable, because seemingly minor textual details—punctuation, wording, or line breaks—may contribute to comedic effect. Retaining source metadata is also useful for later analysis of domain differences, duplicate detection, and possible source-specific biases.

Formally, let the merged corpus be denoted

$$\mathcal{J} = \{(i, y_i, s_i)\}_{i=1}^{N},$$

where y_i is the joke text, i is a global identifier, and s_i collects source metadata. This corpus is the common starting point for all later stages.

2.3 Dense embedding generation

The next stage embeds each joke in a dense semantic vector space. This is implemented in `src/pipelines/embeddings.py`. The current default configuration uses the model `qwen/qwen3-embedding-8b` with embedding dimensionality 1024. Embeddings are computed asynchronously in batches, written to parquet shards, and stored as pairs (i, e_i), where i is the global joke identifier and $e_i \in \mathbb{R}^{1024}$ is the corresponding embedding.

The role of this stage is twofold. First, the joke embeddings are later used to score candidate keywords by semantic relevance. Second, they define the vector index used for retrieving multi-reference joke sets. In other words, the embedding space is the shared representation on which both keyword extraction and reference retrieval depend.

The checked-in entrypoints currently run this stage on a pilot subset rather than the full corpus. This should be understood as an implementation choice for an early-stage thesis, not as a conceptual limitation of the method.

2.4 Keyword extraction as weak prompt approximation

A major correction to the earlier research plan is that the current repository does *not* yet implement generative prompt inversion from jokes to free-form prompts. Instead, it approximates weak prompts through salient keyword extraction. This stage is implemented in `src/pipelines/keywords.py`.

For each joke y_i, the pipeline first generates a candidate pool of surface n-grams with a `CountVectorizer`-based analyzer. Under the default configuration, this analyzer uses unigrams only, removes English stopwords, and keeps at most 64 candidate terms per joke. Let the resulting candidate set be

$$C(y_i) = \{c_{i1}, \ldots, c_{im}\}.$$

Each candidate term is then embedded with the same embedding family used for jokes. If $e(y_i)$ denotes the joke embedding and $e(c)$ denotes the embedding of a candidate keyword, the relevance score is computed by cosine similarity:

$$s_{\mathrm{rel}}(c, y_i) = \frac{e(c)^\top e(y_i)}{\|e(c)\| \, \|e(y_i)\|}.$$

Selecting only the most similar candidates would often yield redundant keywords. To reduce redundancy, the pipeline applies maximal marginal relevance (MMR), choosing up to three keywords while balancing relevance against diversity. Under the default configuration, the diversity coefficient is 0.7 and the number of retained keywords is 3. The output of this stage is therefore a compact keyword representation

$$K(y_i) = \{k_{i1}, k_{i2}, k_{i3}\},$$

possibly with fewer than three items when the candidate pool is small.

Methodologically, these keywords should not be interpreted as full latent prompts. They are weaker objects: compact semantic handles that capture topical or lexical anchors of the joke. This is a pragmatic approximation. It does not solve the joke-to-prompt inversion problem in a linguistically rich sense, but it yields a tractable intermediate representation from which multiple prompt variants can be constructed.

2.5 Reference construction by keyword grouping and retrieval

2.5.1 Keyword grouping and prompt rendering

The reference-construction stage is implemented in `src/pipelines/references.py`. For each joke-level keyword set $K(y_i)$, the pipeline enumerates all unique keyword groups whose size lies between `min_keywords` and `max_keywords`. Under the default configuration, these values are 1 and 2, so the pipeline constructs all one-keyword and two-keyword groups. If

$$K(y_i) = \{k_1, k_2, k_3\},$$

then the generated groups are

$$\{k_1\},\ \{k_2\},\ \{k_3\},\ \{k_1, k_2\},\ \{k_1, k_3\},\ \{k_2, k_3\}.$$

Each group is rendered into a natural-language prompt using the template

```
Write a joke using the following keyword(s): {keywords}
```

This prompt serves two purposes: it is the textual form later given to generation models, and it is also the object embedded for reference retrieval.

This design creates a controlled spectrum of prompt specificity. Single-keyword prompts are broad and often admit many possible jokes. Two-keyword prompts are more constrained and tend to produce more coherent reference sets. The present implementation intentionally stops at two keywords because larger groups would quickly become over-specific, reducing the number of plausible distinct references per prompt.

2.5.2 Embedding-based reference retrieval

Once prompt strings are formed, the pipeline embeds them and retrieves nearby jokes from a FAISS index built over the full joke-embedding collection. The retrieval index is an IVF Flat inner-product index. Before indexing and search, vectors are normalized, so inner product corresponds to cosine similarity. Under the default configuration, the main parameters are:

- embedding model: `qwen/qwen3-embedding-8b`;
- dimensionality: 1024;
- `faiss_nlist`: 4096;
- `faiss_nprobe`: 32;
- `top_k`: 5;
- `oversample`: 10;
- `min_similarity`: 0.5.

For a rendered prompt x, let $q(x)$ be its normalized embedding and let e_j be the normalized embedding of joke y_j. The retrieval score is then

$$s(x, y_j) = q(x)^\top e_j.$$

The search procedure retrieves more than k candidates, filters out those whose similarity falls below the threshold, and retains the top k remaining items. The corresponding texts are then used as the observed reference set:

$$Y^*(x) = \{y_1, \ldots, y_n\}, \qquad n \leq 5.$$

This stage operationalizes the central object of the thesis: a prompt associated with multiple candidate joke references rather than with a single gold target. Importantly, the retrieved set is not an exhaustively annotated set of all good jokes for x. It is a *retrieved approximation* of that set. This matters both empirically and theoretically. Empirically, retrieved references may contain noise or omit valid alternatives. Theoretically, the incompleteness of $Y^*(x)$ becomes a core part of the later MRVF analysis rather than a secondary inconvenience.

A practical consequence of the current implementation is that some keyword groups may retrieve no reference passing the similarity threshold. Such rows are still representable in the dataset because candidate generation uses the prompt itself, not the reference texts. However, rows with empty reference sets would need to be filtered or specially handled before actual MRVF training.

2.5.3 Deduplication and split construction

After retrieval, the resulting dataset contains rows of the form

$$(x_i, Y^*(x_i), S(x_i)),$$

where x_i is the keyword-based prompt, $Y^*(x_i)$ is the retrieved reference set, and $S(x_i)$ is the vector of retrieval scores. Different source jokes can generate identical keyword groups, so the repository deduplicates the dataset by the `keywords` field, keeps the first occurrence, and reassigns row identifiers.

The deduplicated dataset is then split into train, validation, and test partitions. The current implementation uses a two-stage random split with `test_fraction`=0.1, `validation_fraction`=0.1, and a fixed random seed of 42. The result is a standard 80/10/10 partition of the keyword-prompt dataset. Separate parquet shards are written for the full dataset and for each split.

2.6 Candidate generation pipeline

The repository already supports baseline generation over the reference dataset. This stage is implemented in `src/pipelines/candidates.py`. For a selected model and dataset split, the pipeline iterates over the prompt rows and generates one candidate joke per row using a chat-completion API. The generation prompt is the same prompt template used in reference construction:

```
Write a joke using the following keyword(s): {keywords}
```

Under the default configuration, candidate generation uses temperature 1.0 and allows up to 128 completion tokens.

The output schema contains four fields: row identifier, keyword group, model name, and generated text. Candidate files are stored under a per-model, per-split directory structure. This organization is important for later evaluation, because it allows outputs from several models to be aligned by the same underlying prompt row.

Methodologically, this candidate stage should be understood as a benchmarking layer rather than as the core contribution of the thesis. It provides the

artifacts needed to compare baseline models, prompt variants, and future MRVF-trained checkpoints once those checkpoints exist.

2.7 Implemented automatic evaluation

2.7.1 Pairwise comparison setup

Automatic evaluation is implemented in `src/pipelines/evaluation.py`. The pipeline first concatenates candidate datasets from multiple models. It then groups candidates by reference-set identifier and model name, and constructs pairwise comparisons across all model pairs that produced outputs for the same prompt. Left-right ordering is randomized with a fixed seed in order to reduce position bias.

The evaluation prompt is deliberately simple. The user-facing prompt contains the shared joke prompt together with a left and a right candidate. The system instruction tells the evaluator to act as an expert judge of humor quality, choose which joke is funnier for a general audience, avoid ties, and return strict JSON. Under the default configuration, the judge model runs at temperature 0.

This evaluation layer captures only one dimension of quality: comparative funniness under a shared prompt. It does *not* yet implement the fuller multidimensional evaluation proposed elsewhere in the thesis, such as separate rubric-based judgments for prompt adherence, novelty, human-likeness, and funniness. Nor does it yet include human annotation. Accordingly, the current automatic evaluator should be described as the implemented pilot benchmark, not as the final evaluation framework.

2.7.2 Leaderboard aggregation

Pairwise judgments are aggregated into a leaderboard with wins, losses, number of comparisons, win rate, and Bradley–Terry scores. Let $\mathcal{M}$ denote the set of evaluated models. For every judged pair (m_a, m_b), the evaluator records a binary win for one side. The pipeline then estimates Bradley–Terry strengths and reports their logarithms as `bt_score` values.

This aggregation is a sensible choice for humor because relative judgments are often more stable than absolute scores. Asking a judge to decide which of two

jokes is funnier is usually easier than asking for a well-calibrated numerical rating. At the same time, this procedure inherits the limitations of the judge model. Since humor judgments by language models can diverge from human judgments, the current leaderboard should be interpreted as an efficient automatic proxy rather than as a definitive measure of humorous quality.

2.8 Theoretical framework for MRVF training

The implemented pipeline above motivates the theoretical part of the methodology. It constructs the observed multi-reference sets needed by verifier-free reinforcement learning, even though the optimization stage has not yet been coded.

2.8.1 Notation

Let

- x denote a prompt,
- z denote a reasoning trace,
- y denote the final generated joke,
- y^* denote a single reference answer,
- $Y^*(x) \subset \mathcal{Y}$ denote the observed set of acceptable references for prompt x.

The policy is factorized as

$$\pi_\theta(z, y \mid x) = \pi_\theta(z \mid x)\, \pi_\theta(y \mid x, z).$$

This factorization is conceptually useful even if the underlying model emits a single token sequence, because it separates the reasoning process from the final answer event that receives reward.

2.8.2 RL with verifiable rewards

In the standard single-reference, verifiable setting, reward is given by

$$R(y, y^*) = \mathbb{I}\{y = y^*\}.$$

The objective is

$$J_{\mathrm{RLVR}}(\theta \mid x, y^*) = \mathbb{E}_{z\sim\pi_\theta(z|x)}\mathbb{E}_{y\sim\pi_\theta(y|x,z)}[R(y, y^*)].$$

Using the score-function identity,

$$\nabla_\theta \, \mathbb{E}_{u\sim p_\theta(u)}[f(u)] = \mathbb{E}_{u\sim p_\theta(u)}\big[f(u)\nabla_\theta \log p_\theta(u)\big],$$

we obtain

$$\nabla_\theta J_{\mathrm{RLVR}} = \mathbb{E}_{z,y}\Big[R(y, y^*)\big(\nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log \pi_\theta(y \mid x, z)\big)\Big].$$

2.8.3 VeriFree

When an external verifier is unavailable but a single reference answer y^* exists, the expectation over the indicator can be rewritten as

$$\mathbb{E}_{y\sim\pi_\theta(\cdot|x,z)}[\mathbb{I}\{y = y^*\}] = \pi_\theta(y^* \mid x, z).$$

This yields the verifier-free objective

$$J_{\mathrm{VF}}(\theta \mid x, y^*) = \mathbb{E}_{z\sim\pi_\theta(z|x)}\big[r_\theta(x, z, y^*)\big],$$

where

$$r_\theta(x, z, y^*) := \pi_\theta(y^* \mid x, z).$$

Differentiating gives

$$\nabla_\theta J_{\mathrm{VF}} = \mathbb{E}_{z\sim\pi_\theta(z|x)}\Big[r_\theta(x, z, y^*)\big(\nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log \pi_\theta(y^* \mid x, z)\big)\Big].$$

2.8.4 Multi-reference VeriFree

Humor violates the single-reference assumption. A weak prompt such as "write a joke using the keywords prophet, atheism" admits many plausible completions. The implemented repository makes this explicit by constructing multiple retrieved references for the same prompt. Accordingly, reward should be defined at the level

of a reference set:

$$R(y, Y^*) = \mathbb{I}\{y \in Y^*\}.$$

The corresponding objective is

$$J_{\mathrm{MR}}(\theta \mid x, Y^*) = \mathbb{E}_z \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x, z)}[\mathbb{I}\{y \in Y^*\}].$$

Since

$$\mathbb{E}_y[\mathbb{I}\{y \in Y^*\}] = \sum_{y \in Y^*} \pi_\theta(y \mid x, z),$$

we define the multi-reference verifier-free reward

$$r_\theta(x, z, Y^*) := \sum_{y \in Y^*} \pi_\theta(y \mid x, z),$$

and obtain

$$J_{\mathrm{MRVF}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z \mid x)}\big[r_\theta(x, z, Y^*)\big].$$

This objective is the theoretical counterpart of the implemented reference-retrieval pipeline: the code constructs observed multi-reference sets, and MRVF defines how those sets would be used for optimization.

2.8.5 Incomplete observed references

In the present setting, $Y^*(x)$ is not an exhaustive gold set. It is a retrieved subset of jokes judged similar to the prompt in embedding space. Therefore, incompleteness is built into the methodology from the outset. Suppose an observed subset $Y \subset Y^*$ is used in practice. Then

$$r_\theta(x, z, Y) = \sum_{y \in Y} \pi_\theta(y \mid x, z).$$

If each member of Y^* is included with probability ρ, then

$$\mathbb{E}_Y[r_\theta(x, z, Y)] = \rho\, r_\theta(x, z, Y^*).$$

Thus the observed reward is proportional in expectation to the full-set reward. This preserves the objective up to scale, but it does not remove the false-negative

problem: probability mass shifted toward observed references must come from somewhere, and that somewhere includes both bad jokes and unseen good jokes.

To reduce this risk, the proposed optimization includes a KL penalty relative to a reference policy:

$$\max_{\theta} \; \mathbb{E}_{x,z}\big[r_\theta(x, z, Y^*(x))\big] - \beta\, \mathbb{E}_x\Big[\mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big)\Big].$$

The purpose of this term is not merely regularization in the abstract. In a creative domain with incomplete references, it explicitly protects against overly aggressive reallocation of probability mass away from plausible but unobserved jokes.

2.8.6 Proposed training loop

Although `training.py` is currently empty, the intended update rule can be specified. For a prompt x, sample a group of reasoning traces

$$z_1, \ldots, z_G \sim \pi_\theta(z \mid x),$$

and compute the corresponding rewards

$$\hat{r}_i = r_\theta(x, z_i, Y^*(x)).$$

For the score-function term over reasoning traces, a leave-one-out baseline may be used:

$$A_i = \hat{r}_i - \frac{1}{G-1} \sum_{j \neq i} \hat{r}_j.$$

This yields the unbiased reasoning-gradient estimator

$$g_z = \frac{1}{G} \sum_{i=1}^{G} A_i \, \nabla_\theta \log \pi_\theta(z_i \mid x).$$

The reference-probability contribution is

$$g_r = \frac{1}{G} \sum_{i=1}^{G} \nabla_\theta r_\theta(x, z_i, Y^*(x)),$$

which can equivalently be written in log-derivative form wherever $r_\theta(x, z_i, Y^*(x)) > 0$. The full update then adds the KL term:

$$\nabla_\theta J_{\mathrm{MRVF}} \approx g_z + g_r + g_{\mathrm{KL}}.$$

The practical importance of the implemented data pipeline is now clear. Without constructing prompt-conditioned multi-reference sets, the reward $r_\theta(x, z, Y^*(x))$ cannot be estimated. The software pipeline is therefore not auxiliary to the theory. It is the operational precondition for it.

2.9 Evaluation protocol for the preliminary thesis

The present repository supports one implemented benchmark and one broader planned benchmark.

The *implemented benchmark* is automatic pairwise comparison of generated jokes across models, followed by Bradley–Terry aggregation. This is the method that can be honestly reported as operational at the current stage.

The *planned benchmark* extends evaluation along multiple axes motivated by recent humor-evaluation work: prompt adherence, novelty, human-likeness, and funniness. These dimensions should ideally be assessed with calibrated rubrics and with a combination of human annotation and LLM-based evaluation. That extension is consistent with the literature review and with the theoretical goals of the project, but it is not yet implemented in the checked-in code.

Accordingly, the methodological contribution of this preliminary thesis is twofold. On the engineering side, it demonstrates a working pipeline that converts raw joke corpora into weak prompts with retrieved multi-reference sets and supports baseline generation plus automatic comparison. On the theoretical side, it formalizes how these constructed reference sets can be used in a verifier-free reinforcement-learning objective tailored to creative generation.

Chapter 3

Results

3.1 Overview of the implemented pilot

The main result of the current project stage is not a trained humor model. It is a working pilot pipeline that operationalizes the thesis idea at the data and evaluation levels. This result is more substantial than it may initially appear. In a non-verifiable domain, data construction is not a trivial preprocessing step; it is part of the method itself. If the data representation is wrong, the reinforcement-learning objective will optimize the wrong behavior.

The implemented pilot already demonstrates five important facts:

1. large-scale joke aggregation is feasible and already completed;
2. embedding-based retrieval can be used to build sets of references for weak prompts;
3. prompt construction can be approximated through keyword extraction;
4. pairwise generation and evaluation infrastructure is available for future model benchmarking;
5. the resulting artifacts expose concrete limitations that can guide the next implementation stage.

For a preliminary thesis, this is an acceptable and useful stopping point. It provides enough implemented material for a methodology chapter and enough empirical artifacts for a first results chapter, while still leaving room for a final thesis to deliver the actual reinforcement-learning experiments.

3.2 Pilot dataset statistics

Table 3.1 summarizes the key counts extracted from the local repository state. These numbers are especially important because they show the difference between the total available corpus, the current embedded retrieval pool, the seed set used for keyword extraction, and the final deduplicated prompt-reference dataset.

Table 3.1: Pilot data-construction statistics

Artifact	Count
Raw merged joke corpus	664,048
Embedded jokes in current local pilot	10,005
Keyword-annotated jokes in current pilot	1,005
Generated prompt-reference rows before deduplication	5,958
Unique prompt-reference rows after deduplication	5,058
Training prompts	4,046
Validation prompts	506
Test prompts	506

Several observations follow directly from these numbers.

First, the pipeline expands a relatively small seed subset into a much larger prompt-level dataset. From 1,005 keyword-annotated jokes, it produces 5,958 candidate prompt rows. This expansion is expected because each joke contributes multiple one-keyword and two-keyword combinations.

Second, deduplication removes 900 duplicate prompt rows. This is a useful sign rather than a problem. It indicates that different jokes frequently converge to the same weak prompt, which is exactly the condition needed for multi-reference learning.

Third, the size difference between the embedded retrieval pool (10,005 jokes) and the keyword seed set (1,005 jokes) reveals the current pilot strategy: a smaller set of seed jokes is used to generate prompts, but references are retrieved from a larger pool. This is methodologically sensible because it maximizes reference diversity without requiring the full prompt-construction pipeline to run over the entire corpus.

3.3 Reference-set quality

The most important question for the current thesis is whether the retrieved reference sets are large and coherent enough to justify a multi-reference objective. Table 3.2 reports descriptive statistics derived from the local prompt-reference artifacts.

Table 3.2: Descriptive statistics of deduplicated prompt-reference sets

Statistic	Value
Average references per prompt	4.07
Median references per prompt	5
Maximum references per prompt	5
Prompts with zero retrieved references	204
Prompts with exactly one reference	398
Prompts with five references	3,450
Prompts built from one keyword	2,117
Prompts built from two keywords	2,941
Average keyword-group size	1.58

These results are encouraging. The median prompt already reaches the configured reference cap of five items, and 3,450 out of 5,058 prompts are saturated at that limit. This indicates that the retrieval mechanism is not merely finding occasional neighbors; for a large fraction of prompts it constructs a dense local neighborhood of jokes that are semantically related enough to pass the similarity threshold.

At the same time, the long tail is visible. There are 204 prompts with zero retained references and 398 with only one reference. This matters for the later training stage because MRVF becomes weak or unstable when the prompt has no meaningful reference set. The current pilot therefore suggests a natural filtering or curriculum strategy: future training should probably begin with prompts that have at least two or three references and only later decide how to incorporate sparse cases.

The relative abundance of two-keyword prompts is also informative. In the deduplicated dataset, 2,941 prompts are built from two keywords and 2,117 from one keyword. This suggests that the current parameter choice (`min_keywords`=1, `max_keywords`=2) yields a balanced mixture of broad and moderately constrained

prompts. The one-keyword prompts increase diversity; the two-keyword prompts increase specificity.

3.4 Qualitative examples

Quantitative counts alone do not show whether the constructed prompts are semantically meaningful. For that reason, qualitative inspection of pilot rows is necessary. Table 3.3 provides representative examples drawn from the current training split.

Table 3.3: Examples of prompt-reference sets from the pilot dataset

Keyword group	Number of references	Illustrative reference behavior
`[alphabet, does]`	5	References cluster around alphabet-based wordplay, including pirate-letter jokes and orthographic misunderstandings. The prompt is weak, yet the reference set still shares a recognizably playful mechanism.
`[typo, ate]`	5	The references include jokes about autocorrect, eating, and lexical distortion. This is a good example of a prompt that is broad enough to admit multiple answers but still constrained by a recoverable comic theme.
`[ears]`	5	The retrieved references revolve around hearing or cornfield homophones. This shows that even a one-keyword prompt can generate a coherent neighborhood when the word carries a strong pun affordance.

Keyword group	Number of references	Illustrative reference behavior
`[cider, favourite]`	2	The prompt retrieves a much narrower set. This reveals the risk of under-supported prompts: they may be too idiosyncratic for robust training even if they remain semantically meaningful.
`[girl, lazy]`	1	The prompt effectively collapses to a near-singleton neighborhood. Such rows are useful for evaluation but weak for multi-reference reinforcement learning.

These examples support three conclusions.

First, the retrieval mechanism often preserves a plausible humor mechanism even though it is not explicitly told to search for mechanisms. This is a positive sign for the embedding-based approach.

Second, one-keyword prompts can still be useful when the keyword is semantically rich or strongly associated with punning. This means that prompt broadness is not automatically a weakness.

Third, sparsity remains a real issue. Some prompts are clearly under-supported, which means that future training cannot simply treat every prompt row as equally informative.

3.5 Interpretation of the pilot as a multi-reference dataset

The pilot results support the central methodological claim of the thesis: a joke corpus can indeed be restructured from isolated samples into weak prompts paired with multiple references. This is not merely a convenience for evaluation. It changes the shape of the learning problem.

In a standard SFT dataset, the model sees (x, y^*) pairs and is asked to imitate the observed answer. In the pilot dataset produced here, the learning

problem begins to look like $(x, Y^*(x))$, where the goal is not to reproduce one answer exactly, but to shift probability mass toward a set of acceptable answers. That is precisely the structure required by the MRVF objective.

The descriptive statistics also make the conceptual advantages of multi-reference modeling concrete. If the majority of prompts already have several references in a small pilot, then a larger-scale run over the full corpus has a reasonable chance of yielding a much richer multi-reference dataset. The pilot therefore functions as an existence proof for the data representation.

3.6 Limitations revealed by the current implementation

The pilot also reveals several limitations that must be acknowledged explicitly.

3.6.1 Prompt quality is approximate

The current system does not yet perform full prompt inversion from jokes. It uses keywords as a proxy. This is computationally efficient and easy to audit, but it inevitably loses structure. A keyword prompt such as `Write a joke using the following keyword(s): ears` can recover the topic of a joke, but not necessarily its framing, rhetorical setup, or intended comedic mechanism.

This limitation does not invalidate the pilot. It means that the current prompt representation should be interpreted as a *weak constraint*, not a rich semantic specification. That interpretation is consistent with the thesis goal: the training task should preserve multiplicity of valid outputs. Weak prompts may even be preferable for that reason, provided they remain coherent enough for retrieval.

3.6.2 Coverage is still partial

Only a subset of the total corpus has been embedded, and an even smaller subset has been used for keyword extraction in the current local run. Therefore, the reported numbers are pilot indicators, not final corpus-scale statistics. A final thesis should rerun the pipeline on a much larger portion of the corpus before

drawing strong conclusions about reference coverage.

3.6.3 Reference incompleteness remains unresolved

Even when the system finds five references, that does not mean the true answer set has been observed fully. Five is only the configured retrieval cap. The present pilot therefore confirms the relevance of the incompleteness problem discussed in the methodology chapter. The retrieved set is useful, but it should still be interpreted as a partial observation of a larger latent set of good jokes.

3.6.4 No committed benchmark outputs yet

The repository contains implemented candidate-generation and evaluation modules, but the current committed data artifacts do not yet include candidate generations or pairwise evaluation results. This means the present thesis can defend the benchmark design, but not yet report final model rankings. That limitation is acceptable in a preliminary thesis, provided it is stated clearly.

3.6.5 Training remains future work

The most important limitation is also the most obvious one: the actual reinforcement-learning stage has not yet been implemented. The theoretical objective is specified, and the data pipeline now justifies it, but a final thesis must still test whether MRVF training improves humor generation relative to base, instruct, and reasoning checkpoints.

3.7 What the pilot already contributes

Despite those limitations, the current work already contributes something non-trivial.

First, it replaces a purely conceptual report with a thesis-grade research narrative grounded in actual code and actual data artifacts. The methodology is no longer hypothetical in the abstract; major components are implemented and inspectable.

Second, it demonstrates that the move from single-reference humor supervision to multi-reference supervision is practically achievable. This is arguably the main precondition for the later training study.

Third, it clarifies the software architecture needed for the final stage. The project now has separate modules for corpus ingestion, representation learning, prompt construction, retrieval, generation, and evaluation. This modularity reduces the risk that future negative results will be uninterpretable because all components were entangled.

Fourth, it identifies the exact frontier between what has already been validated and what still requires experimentation. That clarity is itself valuable in a thesis context. It prevents overclaiming and allows the final version of the thesis to focus its experimental effort where uncertainty is highest.

3.8 Next stage of the study

Based on the current pilot, the next stage of the project should proceed in the following order.

First, the prompt-construction pipeline should be rerun at a larger scale so that the distribution of reference counts becomes more stable. A final thesis should not rely on only 1,005 keyword-seeded jokes if the full corpus is available.

Second, sparse prompts should be filtered or stratified. Prompts with zero or one retrieved reference are useful diagnostic cases, but they are poor anchors for multi-reference reward estimation.

Third, the `training.py` module should implement the MRVF update with careful variance control and KL regularization.

Fourth, model comparisons should be run over at least four checkpoint families: base, instruction-tuned, reasoning-tuned, and MRVF-trained variants. This matches the experimental sketch of the original report but relocates it to the correct stage of the project.

Fifth, automatic pairwise evaluation should be complemented with a smaller but carefully designed human annotation study. The literature strongly suggests that humor cannot be left entirely to model judges [Sakabe et al., 2025, Zhang et al., 2024].

In short, the pilot has done the preparatory work needed to make a fi-

nal reinforcement-learning experiment worthwhile. It has not yet replaced that experiment.

Conclusion

This thesis addressed humor generation as a non-verifiable reasoning problem. The central claim was that training for humor should move away from single-reference imitation and toward multi-reference probability allocation over weak prompts. The literature review showed why this shift is needed: SFT is prone to imitation and memorization, preference-based RL is unstable in humor due to noisy judgments, and current multistep humor systems improve quality at the cost of hard-coded reasoning structures.

The work made two connected contributions. At the methodological level, it formulated a multi-reference extension of verifier-free reinforcement learning in which the reward is the total probability mass assigned to a set of acceptable references rather than to one gold answer. At the implementation level, it built and analyzed a pilot pipeline that turns a large joke corpus into prompt-reference sets through embeddings, keyword extraction, vector retrieval, deduplication, and data splitting.

The empirical analysis of the pilot showed that the idea is viable. In the current local repository state, a corpus of 664,048 jokes has already been assembled, a pilot subset has been embedded and processed, and the resulting workflow produces 5,058 deduplicated prompt-reference rows with an average of 4.07 references per prompt and a median of five. These statistics indicate that the core structural requirement of MRVF learning, namely the existence of prompts with multiple acceptable references, can be satisfied in practice.

At the same time, the thesis remains preliminary in the correct sense. The prompt-construction procedure is still approximate, the downstream benchmark artifacts are not yet committed, and the MRVF training loop itself has not yet been implemented. Therefore, the present work should be read as a thesis foundation rather than as a final empirical verdict on reinforcement learning for humor.

The main conclusion is that the project has now crossed the boundary from a speculative report to a defensible research program. The data representation,

software pipeline, and formal objective are aligned. The next step is to implement the training stage and test whether the constructed multi-reference dataset can indeed elicit better reasoning and stronger humor than standard post-training baselines.

References

Mostafa Amin and Manuel Burghardt. A survey on approaches to computational humor generation. In *Proceedings of the 4th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 29–41, 2020. URL https://aclanthology.org/2020.latechclfl-1.4/.

Yahui Chen, Buxin Shi, and Meng Si. Prompt to GPT-3: Step-by-step thinking instructions for humor generation, 2023. URL https://arxiv.org/abs/2306.13195. arXiv:2306.13195.

Tianyu Chu, Shuhua Zhang, Yu Chen, Ning Ding, and Tao Yu. SFT memorizes, RL generalizes: A comparative study of foundation model post-training, 2025. URL https://openreview.net/forum?id=2P4rYTtGok. OpenReview / arXiv preprint.

Andrea Cocchieri, Lorenzo Ragazzi, Pietro Italiani, Giuseppe Tagliavini, and Gianluca Moro. "what do you call a dog that is incontrovertibly true? dogma": Testing LLM generalization through humor. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22922–22937, 2025. URL https://doi.org/10.18653/v1/2025.acl-long.1117.

Victor De Marez, Tim Winters, and Ayla Rigouts Terryn. THInC: A theory-driven framework for computational humor detection, 2024. URL https://arxiv.org/abs/2409.01232. arXiv:2409.01232.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL https://doi.org/10.48550/arXiv.2501.12948. arXiv:2501.12948.

Leo Gao, John Schulman, Jacob Hilton, et al. Scaling laws for reward model overoptimization, 2023. URL https://arxiv.org/abs/2210.10760. arXiv:2210.10760.

Hamid Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme, and Chris Kedzie. LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://aclanthology.org/2024.acl-long.745/.

Justine T. Kao, Roger Levy, and Noah D. Goodman. A computational model of linguistic humor in puns. *Cognitive Science*, 40(5):1270–1285, 2016. URL https://doi.org/10.1111/cogs.12269.

Sangho Kim and Lydia B. Chilton. AI humor generation: Cognitive, social and creative skills for effective humor, 2025. URL https://arxiv.org/abs/2502.07981. arXiv:2502.07981.

Jens Lemmens and Victor De Marez. Computational humor modeling: A survey on the state of the art. *ACM Computing Surveys*, 58(7):177:1–177:37, 2026. URL https://www.uantwerpen.be/en/staff/jens-lemmens/publications/.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050. OpenAI / arXiv preprint.

Thomas Loakman, William Thorne, and Chien-Sheng Lin. Who's laughing now? an overview of computational humour generation and explanation, 2025a. URL https://arxiv.org/abs/2509.21175. arXiv:2509.21175.

Thomas Loakman, William Thorne, and Chien-Sheng Lin. Comparing apples to oranges: A dataset & analysis of LLM humour understanding from traditional puns to topical jokes. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9502–9518, 2025b. URL https://doi.org/10.18653/v1/2025.findings-emnlp.505.

Graeme Ritchie. Current directions in computational humour. *Artificial Intelligence Review*, 16(2):119–135, 2001. URL https://doi.org/10.1023/A:1011610210506.

Ryo Sakabe, Hwiyeon Kim, Tatsuya Hirasawa, and Mamoru Komachi. Assessing the capabilities of LLMs in humor: A multi-dimensional analysis of Oogiri generation and evaluation, 2025. URL https://arxiv.org/abs/2511.09133. arXiv:2511.09133.

Alexey Tikhonov and Pavel Shtykovskiy. Humor mechanics: Advancing humor generation with multistep reasoning, 2024. URL https://arxiv.org/abs/2405.10073. arXiv:2405.10073.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022. URL https://doi.org/10.48550/arXiv.2201.11903. arXiv:2201.11903.

Orion Weller and Kevin Seppi. The rJokes dataset: A large scale humor collection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 6136–6141, 2020. URL https://aclanthology.org/2020.lrec-1.753/.

Tim Winters and Sam Van der Stockt. Evaluating humor generation in an improvisational comedy setting. *Computational Linguistics in the Netherlands Journal*, 14:505–523, 2025. URL https://www.clinjournal.org/clinj/article/view/214.

Zhen Xu, Shaojie Yuan, Li Chen, and Diyi Yang. "a good pun is its own reword": Can large language models understand puns? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11766–11782, 2024. URL https://doi.org/10.18653/v1/2024.emnlp-main.657.

Tianyu Yu, Bin Ji, Shun Wang, Shunyu Yao, Zhijian Wang, Ganqu Cui, Luqi Yuan, Ning Ding, Yong Yao, Zhiyuan Liu, Maosong Sun, and Tat-Seng Chua. RLPR: Extrapolating RLVR to general domains without verifiers, 2025. URL https://doi.org/10.48550/arXiv.2506.18254. arXiv:2506.18254.

Jiawei Zhang, Sizhe Luo, Ruijie Zhang, and Qi Su. HUMORCHAIN: Theory-guided multi-stage reasoning for interpretable multimodal humor generation, 2025. URL https://doi.org/10.48550/arXiv.2511.21732. arXiv:2511.21732.

Jie Zhang, Lily Jain, Yichi Guo, Justin Chen, Kexin Linda Zhou, Sreeranjani Suresh, Amanda Wagenmaker, Scott Sievert, Taylor Rogers, Kevin Jamieson, Robert Mankoff, and Robert Nowak. Humor in AI: Massive scale crowd-sourced preferences and benchmarks for cartoon captioning, 2024. URL https://arxiv.org/abs/2406.10522. Advances in Neural Information Processing Systems, Datasets and Benchmarks Track.

Shaoxiong Zhong, Zicheng Huang, Shufan Gao, Wen Wen, Lin Lin, Marinka Zitnik, and Pan Zhou. Let's think outside the box: Exploring leap-of-thought in large language models with creative humor generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13246–13257, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Zhong_Lets_Think_Outside_the_Box_Exploring_Leap-of-Thought_in_Large_Language_CVPR_2024_paper.html.

Xingcheng Zhou, Zihan Liu, Ariel Sims, Han Wang, Tianyu Pang, Chao Li, Li Wang, Min Lin, and Chao Du. Reinforcing general reasoning without verifiers, 2025. URL https://arxiv.org/abs/2505.21493. arXiv:2505.21493.

Appendix A

Implemented Prompt Templates

This appendix records the main templates already implemented in the repository. They are included here because they define the operational form of the prompts used throughout the pilot.

A.1 Reference-generation prompt

The prompt used for reference retrieval and candidate generation is:

```
Write a joke using the following keyword(s): {keywords}
```

Its main advantage is consistency. The same textual form is used for retrieval-time prompt embedding and for later candidate generation, which reduces mismatch between dataset construction and evaluation. Its main disadvantage is that it is intentionally generic and therefore does not encode humor theory or audience information.

A.2 Embedding query instructions

Two short instruction templates are used to adapt the embedding model to retrieval subtasks:

```
Instruct: Given a keyword for a joke, retrieve jokes
would match the keyword.
Query: {keyword}
```

```
Instruct: Given a prompt for a joke, retrieve jokes
that would answer the prompt.
Query: {prompt}
```

These templates are important because the pilot does not use raw keyword strings as embeddings directly. It embeds them inside retrieval-oriented instructions, which makes the semantic intent of the vector space more task-specific.

A.3 Evaluation prompt

The pairwise evaluation system prompt asks the judge to compare two jokes written for the same prompt and choose exactly one winner. The instruction explicitly discourages rewarding length or polish when these do not improve humor. This design is intentionally simple and should be viewed as a baseline evaluation protocol rather than a final human-aligned rubric.

Appendix B

Supplementary Notes on the Software Architecture

The repository is organized as a set of asynchronous pipelines:

- `JokesPipeline`: downloads and preprocesses raw corpora;
- `EmbeddingsPipeline`: computes dense joke embeddings;
- `KeywordsPipeline`: extracts salient keywords by similarity and MMR;
- `ReferencesPipeline`: builds prompt-reference datasets through vector retrieval;
- `CandidatesPipeline`: generates jokes from prompts;
- `EvaluationPipeline`: performs pairwise judging and leaderboard aggregation.

This architecture matters for the final thesis because it separates data construction from model comparison. As a result, the later MRVF training stage can be added without rewriting the rest of the infrastructure. The missing component is precisely the one that now becomes the main target of future work: a training pipeline that consumes the constructed prompt-reference dataset and optimizes the formal objective introduced in Chapter 2.